Object Detection and Navigation Assistance for Visually Impaired Individuals

VisionAssist AI — Model Selection & System Architecture Report

Undergraduate thesis prototype

Every figure in this report was measured on the project's own hardware and dataset. Nothing is quoted from published benchmarks.

Hardware: NVIDIA GeForce RTX 3050 Ti Laptop (4 GB) · CUDA 12.8 · Windows 11

Software: Python 3.13.14 · PyTorch 2.11.0+cu128 · Ultralytics 8.4.84 · OpenCV 4.13.0

1. Executive summary

The system answers one question continuously: *what is in front of the user, where is it, and does it need to be spoken about?* That splits into two very different problems, and they are solved by two very different kinds of component.

Perception is learned. A convolutional neural network detects objects, because no hand-written rule can recognise a rickshaw from pixels. **Navigation is not learned.** Deciding which obstacle matters, and whether to speak, is a deterministic rule engine — no neural network, no training data, no inference cost.

That division is the central design decision of this project, and Section 4 explains why a learned navigation policy would have been the wrong choice here.

2. Which models are used

Stage	Model / method	Type	Role
Object detection (primary)	`YOLO11n` fine-tuned → `best.pt`	TRAINED	The thesis model. 9 outdoor footpath classes, trained on this project's annotated Dhaka dataset.
Object detection (indoor)	`YOLO11n` COCO-pretrained	PRE-TRAINED	Runs alongside the thesis model, contributing 12 indoor furniture classes only.
Annotation (offline)	`YOLOv8x-WorldV2` (YOLO-World, open-vocabulary)	OFFLINE	Generated candidate boxes for human review. **Not part of the running system.**
Spatial analysis	Geometric — bounding-box centroid & area	RULE	LEFT / CENTER / RIGHT and NEAR / MEDIUM / FAR.
Navigation	Deterministic priority + event engine	RULE	**No ML model.** Decides what to say and when to stay silent.
Language generation	Template grammar (EN + BN)	RULE	**No LLM.** Deterministic sentence construction.
Speech (Bengali)	`edge-tts` → `bn-BD-NabanitaNeural`	NEURAL TTS	Microsoft neural voice. Requires internet.
Speech (English)	`pyttsx3` → Windows SAPI5	SYSTEM TTS	Offline, zero latency.
Depth / distance	— none —	ABSENT	No metric distance is claimed anywhere. See §7.

> **The short answer.** Detection uses **YOLO11n**, fine-tuned on this thesis's own annotated data. Navigation uses **no model at all** — it is an explainable rule engine. Speech uses a pre-trained neural TTS voice for Bengali.

3. Why YOLO11n for detection

3.1 Why a single-stage detector

The application must react before the user's next step. A two-stage detector (Faster R-CNN and relatives) proposes regions and then classifies them, which is accurate but too slow for a navigation aid on a laptop GPU. A single-stage detector predicts boxes and classes in one forward pass, which is what makes real-time operation possible at all.

3.2 Why the *nano* variant

The constraint is 4 GB of VRAM, shared with the display. Measured on this machine at 640 px:

Model	Parameters	File size	Latency	Throughput
`best.pt` (thesis, YOLO11n)	2,591,595	5.5 MB	19.0 ms	52.6 FPS
`yolo11n.pt` (COCO, indoor)	2,616,248	5.4 MB	18.1 ms	55.3 FPS

Both models together still leave headroom for the camera thread, JPEG encoding and the web server, and the observed end-to-end rate is **25–31 FPS** in live camera mode. A larger variant (`yolo11s` / `yolo11m`) would raise accuracy but would not fit the latency budget alongside a second model — and on this dataset, accuracy is limited by *annotation*, not by capacity (§6.3). Spending the compute budget on a bigger backbone would not have fixed the classes that are actually failing.

3.3 Why YOLO11 rather than YOLOv8 or a transformer

- **YOLO11 over YOLOv8** — same Ultralytics API and tooling, slightly better accuracy-per-parameter, and it is the current maintained line.
- **Over DETR / RT-DETR** — transformer detectors need substantially more data and far longer schedules to converge. With ~1,900 training images that is the wrong trade.
- **Over SSD / MobileNet-SSD** — weaker on small and overlapping objects, which is precisely the Dhaka footpath case: crowds, stacked vehicles, partial occlusion.
- **Transfer learning** — training started from COCO-pretrained weights (`pretrained: True`). Training from scratch on 1,934 images would badly underfit.

3.4 Why a second, pre-trained model indoors

The thesis dataset is Dhaka *footpaths*. It contains essentially no indoor imagery — a 30-image sample of the unused pool contained roughly two indoor scenes. Indoor classes therefore **could not honestly be trained** from this data.

Rather than claim indoor capability the data cannot support, the system runs a COCO-pretrained model alongside the thesis model and takes only these classes from it: `chair` , `couch` , `bed` , `dining table` , `tv` , `toilet` , `sink` , `refrigerator` , `oven` , `microwave` , `potted plant` , `bench` .

Two rules keep the fusion sound:

1. The indoor model is **excluded from every class the thesis model covers** (notably `person`), so the two can never double-report one object.
2. On any overlap above **0.55 IoU the thesis model wins**, because it is the model trained on this thesis's data.

> **Declare this in the defence.** The indoor classes are COCO-pretrained and carry **no thesis-measured mAP**. They are a deployment convenience, not a research contribution. `door` is deliberately absent: COCO has no door class, and inventing support for it without annotation would be dishonest.

3.5 Why YOLO-World was used — and why it is not in the running system

The raw dataset of 21,359 images had **zero annotations**. Annotating from scratch is the reason three earlier attempts at this thesis stalled. `YOLOv8x-WorldV2` , an open-vocabulary detector that accepts natural-language prompts, was used **offline** to generate candidate boxes which a human then reviewed.

It produced 24,010 candidates over 2,787 images in 5.6 minutes on the GPU. It is an **annotation aid only** — it never runs at inference time, and the honest methodology phrase is *"machine-assisted annotation with human verification."*

4. Why navigation uses no machine-learning model

This is the question most likely to be asked in a viva, so the reasoning is stated explicitly.

A learned navigation policy — a network that consumes detections and emits an instruction — was considered and **rejected** for four reasons:

1. **No training data exists for it.** Supervising such a model needs thousands of (scene → correct instruction) pairs judged by blind users. That dataset does not exist and collecting it is a thesis in itself.
2. **It must be explainable.** When the system warns about a pothole, it must be possible to say exactly why. A rule engine can be read line by line and defended in a viva; a learned policy can only be characterised statistically.
3. **Failures must be predictable.** This is assistive safety software. A rule engine fails in ways that can be enumerated in advance; a network can fail in novel ways on inputs slightly outside its training distribution.
4. **The rules are genuinely simple.** Priority ordering, an escalation test and a cooldown are a few dozen lines. Replacing that with a network would add training cost, inference cost and opacity while solving nothing.

> **Principle applied throughout:** learn what cannot be specified (recognising a rickshaw from pixels); specify what should not be learned (whether a hole in the ground is more urgent than a chair).

4.1 The priority score

Every detection is scored so the ordering is explainable rather than tuned:

```
score  = 100  if class is a danger class (pothole, open_drain, stairs, curb, pole, vehicle)
       + 40   if class is an indoor obstacle (chair, table, bed, …)
       + 50   if position == CENTER          (it is in the user's path)
       + 30   if proximity == NEAR           (12 if MEDIUM, 0 if FAR)
       + 10 x confidence                     (breaks ties, never outweighs geometry)
```

A pothole ahead therefore always outranks a chair to the left, and geometry always outweighs the detector's own confidence.

4.2 When the system speaks — and when it stays silent

A detector at 25 FPS produces roughly 1,500 detections per minute. Narrating them would make the system unusable, so speech is triggered by **change, not presence**:

- a new important object appears;
- proximity worsens (FAR → MEDIUM → NEAR);
- an object moves into the CENTER zone.

A pole that has been on the left for twenty seconds is not news. On top of this sit a per-object cooldown (default 3.0 s) and a global minimum gap (1.2 s), both configurable. A genuine escalation to NEAR overrides the cooldown, because that is exactly when the user needs to hear it.

> **A subtle but important detail.** The engine compares against the state at the *last announcement*, not the last frame. Comparing against the last frame would let an escalation that happened to be suppressed by the cooldown be consumed and silently lost — and the escalation most likely to be suppressed is MEDIUM → NEAR, the one that matters most.

5. Spatial reasoning

5.1 Direction

```
object_center_x = (x1 + x2) / 2
frac = object_center_x / frame_width

frac < 0.33            → LEFT
0.33 ≤ frac ≤ 0.66     → CENTER
frac > 0.66            → RIGHT
```

Both boundaries are runtime-configurable, not hard-coded constants.

5.2 Proximity — and why it is not distance

```
area_ratio = (box_width x box_height) / (frame_width x frame_height)

area_ratio ≥ 0.15  → NEAR      (কাছে)
area_ratio ≥ 0.04  → MEDIUM
otherwise          → FAR       (দূরে)
```

> **Stated honestly:** this is *apparent-size proximity*, not distance. A bus 30 m away can occupy more pixels than a chair 2 m away, so the estimate is only comparable between objects of similar real-world size. A single RGB camera cannot recover metric distance.
>
> The UI therefore never prints metres. The original interface mock-up showed “2.4 meters”; that was **removed** and replaced with the proximity bucket and zone, because printing a fabricated number in an assistive device is worse than printing nothing.
>
> `spatial.py` defines a `DepthProvider` interface: supplying MiDaS, Depth-Anything, or a stereo/ToF sensor turns proximity into real depth with no other code change.

6. Training and results

6.1 Dataset

Stage	Images	Note
Raw pool	21,359	Mixed sources, **zero annotations**
Dhaka footpath group selected	3,000	Numeric `.jpg`, 256×256
After near-duplicate removal	2,787	dHash, Hamming ≤ 5
Human-verified & exported	2,782	23,962 boxes
Train / Val / Test	1,934 / 553 / 276	70/20/10, seed 42, leakage guard on

De-duplication happens **before** annotation, so the same scene is never labelled twice and a frame cannot appear in train with its near-copy in test — that would inflate mAP and be indefensible.

6.2 Configuration

```
model: yolo11n.pt (COCO-pretrained)   epochs: 100        batch: 8
imgsz: 640                            optimizer: auto    lr0: 0.01 → lrf: 0.01
momentum: 0.937                       weight_decay: 0.0005
patience: 30                          seed: 42           AMP: enabled
augmentation: mosaic 1.0 (off for last 10), fliplr 0.5, rotation ±5°, HSV-V 0.4
device: CUDA:0 (RTX 3050 Ti)
```

Rotation is small and deliberate: a hand-held or chest-mounted camera is rarely level, but a footpath scene is never upside-down.

6.3 Results — held-out test split

Overall: precision 0.645 · recall 0.404 · mAP@50 0.419 · mAP@50-95 0.297

Class	P	R	mAP@50	mAP@50-95	Test boxes
person	0.811	0.749	**0.837**	0.600	878
parked_vehicle	0.758	0.688	**0.781**	0.528	1360
curb	0.413	0.450	**0.359**	0.292	20
street_obstacle	0.404	0.267	**0.295**	0.197	99
pole	0.263	0.222	**0.268**	0.221	9
tree	0.372	0.262	**0.237**	0.138	65
stairs	0.785	1.000	0.995 *	0.697	**2**
pothole	1.000	**0.000**	**0.000**	0.000	**1**
open_drain	1.000	**0.000**	**0.000**	0.000	**1**

These three rows must be read honestly, and volunteered before an examiner finds them.

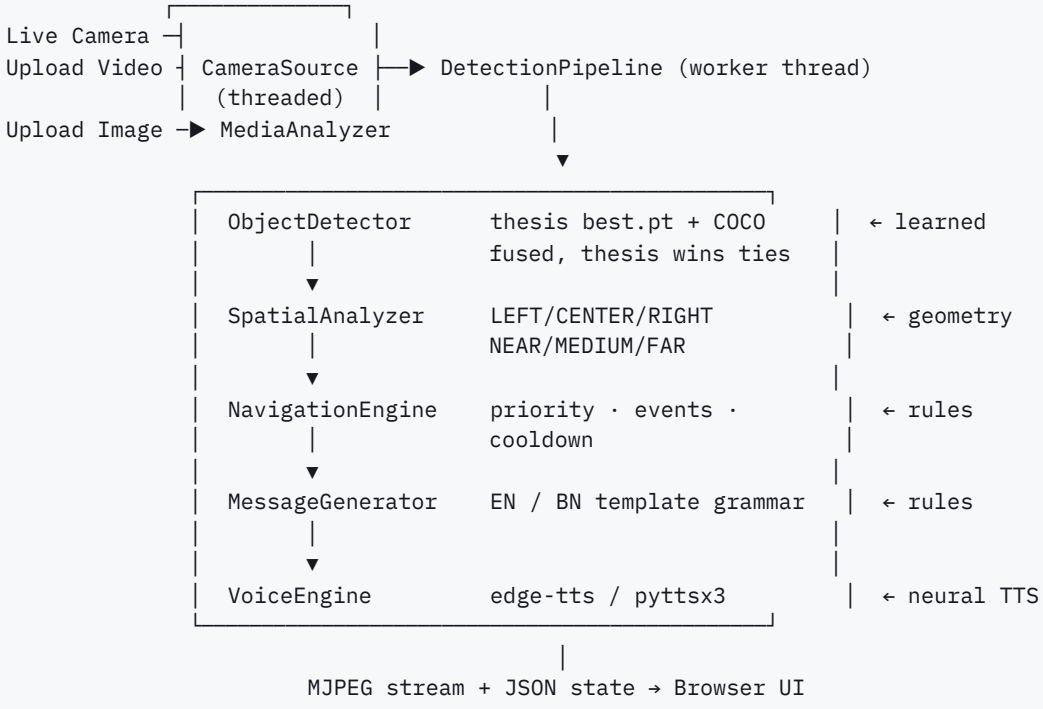
* `stairs` at 0.995 is *not* a good result — it is two test boxes. That is statistical noise, not evidence.

`pothole` and `open_drain` show **zero recall**: the model detects none of them. Their precision of 1.000 is an artefact of never making a positive prediction at all, not a sign of quality.

Cause: they had 17 and 12 training instances respectively. The open-vocabulary annotation aid cannot recognise ground-surface defects — it has no visual prototype for a hole in pavement and cannot separate one from a shadow or a paving seam — so almost no candidate boxes were produced for a human to confirm.

Consequence: the model is genuinely strong on `person` and `parked_vehicle` (thousands of examples each) and genuinely **blind to the fall hazards**. This is an *annotation* limitation, not a training or architecture one: more epochs or a larger backbone will not change it. The fix is to hand-annotate 100–200 instances of each hazard class.

7. System architecture



7.1 Threading

Three threads, deliberately: **camera capture**, the **vision pipeline**, and **speech synthesis**. Voice never blocks inference, inference never blocks HTTP, and the camera always holds the newest frame — for a navigation aid a *fresh* frame matters far more than processing *every* frame. Frames are dropped rather than queued, so the view can never drift seconds behind reality.

7.2 One detection stack, three input modes

An uploaded video becomes the pipeline's **source**, exactly like a webcam. This is why video is processed frame-by-frame as it streams, rather than analysed in full and replayed afterwards: there is only one loop, and it is the live one. Detection, navigation, voice, FPS and the detection table therefore work identically in all three modes, and a fix in one is a fix in all.

8. Bengali language generation

Sentences are constructed by rule, not translated by a model. Two properties of Bengali are handled explicitly:

- **Word order** is Subject–Object–Verb, so sentences are assembled tail-last with আছে closing the clause.
- **Numeral classifiers** — একজন for people, একটি for objects. Using একটি for a person is grammatically wrong and jarring to a native speaker, so animacy is tracked per class.

Situation	English	Bengali
Person, left	There is a person on your left.	আপনার বাম পাশে একজন মানুষ আছে।
Pothole ahead, near	Careful! A pothole is close ahead.	সাবধান! সামনে খুব কাছে একটি গর্ত আছে।
Three objects (combined)	There is a person on your left, a chair ahead and a rickshaw on your right.	আপনার বাম পাশে একজন মানুষ, সামনে একটি চেয়ার এবং আপনার ডান পাশে একটি রিকশা আছে।

Multiple objects become **one sentence**, not three. For a listener who cannot see the scene, a single flowing sentence is quicker to absorb than three restarts of “There is …”, and the single closing আছে is what keeps the Bengali from sounding machine-generated.

8.1 Speech synthesis

Windows SAPI5 on this machine has **no Bengali voice installed**, so `pyttsx3` physically cannot speak Bengali here. Bengali therefore uses `edge-tts` with `bn-BD-NabanitaNeural` (free, no API key, **requires internet**), with the browser Web Speech API as fallback. English uses `pyttsx3` — offline and instant. Installing a Bengali Windows speech pack would make Bengali fully offline with no code change.

9. Known limitations

1. **Fall hazards are not detected.** `pothole` and `open_drain` have zero recall (§6.3). This is the most significant limitation of the system.
2. **Proximity is not distance.** Apparent size only; comparable within a class, not across classes.
3. **Indoor classes are not thesis-trained** and carry no measured mAP. `door` is unsupported.
4. **Bengali speech requires internet** on this machine.
5. **No object tracking.** Guidance keys on (class, position), so two people on the left are one piece of news; a specific object is not followed across frames.
6. **Daylight data only.** The selected images are daytime; night performance is untested.
7. **Single monocular camera.** No stereo, IMU or GPS.

8. **Latency.** 25–31 FPS on a laptop RTX 3050 Ti. A deployable aid would need an embedded target and an end-to-end latency budget measured to the ear, not just to the screen.

10. Running the system

```
cd "k:\University Study\University\Thesis 4"
python app/main.py --port 8010          # then open http://127.0.0.1:8010

python app/main.py --camera 1           # a different camera index
python app/main.py --camera clip.mp4    # a video file
python app/main.py --language en        # start in English (default: Bengali)
python -m backend.camera                # list working camera sources

python scripts/train.py                 # retrain
python scripts/validate.py              # P, R, mAP50, mAP50-95, confusion matrix
```

The application discovers `best.pt` automatically on every start and falls back to the COCO placeholder only when no trained model exists — announcing that fact in the log, on the video overlay and in a UI banner, so a placeholder run can never be mistaken for a thesis result.

VisionAssist AI — Object Detection and Navigation Assistance for Visually Impaired Individuals. Research prototype: this system provides experimental computer-vision assistance and must not be relied upon as the sole navigation aid for real-world mobility. All measurements taken on the project hardware described on page 1.